

AIRCOMMAND-OS

Real-Time Hand Gesture Recognition & OS Control System

Author:

Toheed-Ali

Date of Generation:

May 17, 2026

Repository:

github.com/Toheed-Ali/AirCommand-OS

1. Project Introduction

AirCommand-OS is a state-of-the-art touchless system designed to bridge physical human interaction with system-level controls. By capturing real-time high-speed webcam frames, the system extracts articulable 21-landmark 3D hand joints. A customized deep learning classification network translates these spatial features into exact user intents, dispatching commands such as adjusting speaker volumes, changing screen brightness, handling media players, or executing operations synchronously on a connected Flutter mobile application.

2. System Architecture & Technologies Used

The application maintains a highly optimized pipeline separating spatial coordinates collection, geometric normalisation, fast model inference, and network-centric action dispatching:

- **Google MediaPipe Tasks API:** Processes webcam frames natively at low CPU footprints, yielding raw 21 coordinates in 3D space (X, Y, Z).
- **TensorFlow & Keras Engine:** Hosts the deep network classifier, running prediction gates under 5 milliseconds.
- **Scikit-Learn Infrastructure:** Maintains scalable normalisation scalars and dynamic label encoders, eliminating state leaks.
- **Asynchronous WebSocket Server:** Operates a streaming bridge on port 8765 to feed instant action feedback packets to the Flutter app UI.

3. Dataset Journey & Folder Splits

To achieve highly generalisable classifier metrics, a strict 80-20 dataset division was instituted across all gesture classes. Hand-captured samples containing OneDrive Cloud file attribute states (ReparsePoints) were successfully recovered and segmented safely, protecting the neural network from evaluation leakage.

DATASET SPLIT STATISTICS

- Total Raw Samples Detected: 10333 landmarks vectors
 - Train Split (Raw): 8272 samples (~80%)
 - Test Split (Raw): 2061 samples (~20%)
- Balanced Augmented Training Set: 76800 samples (12,800 per class)
- Final Leak-Free Preprocessed Splits:
 - Train Split (Augmented): 65280 samples (85%)
 - Validation Split (Augmented): 11520 samples (15%)
 - Test Split (Pristine Raw): 2061 samples (100% isolated)
- Gestures Supported: fist, l_gesture, palm, peace, three_fingers, thumbs_up

4. The Geometric Feature Engineering Pipeline

Before feeding landmarks directly to the classifier, raw outputs from MediaPipe undergo a three-stage deterministic transformation, allowing perfect prediction accuracy regardless of user position, distance, or scale:

- **Wrist-Origin Translation:** Shift all 21 joints relative to Landmark 0 (wrist) positioned at origin (0,0,0).
- **Hand-Size Scaling:** Normalizes distance ratios by dividing vector arrays by the Euclidean distance from wrist to Landmark 12 (middle finger tip).
- **Z-Depth Clipping:** Constrains depth noise by clipping Z offsets between [-0.15, 0.15], dampening depth fluctuations.

5. Deep Learning Model Architecture

An optimized feed-forward Multi-Layer Perceptron (MLP) architecture was designed for low latent footprint and high accuracy on resource-constrained platforms:

Neural Network Stack:

- Input Layer: 63 features (21 landmarks × 3 dimensions)
- Layer 1: Dense (256 units, ReLU) + BatchNormalisation + Dropout (30%)
- Layer 2: Dense (128 units, ReLU) + BatchNormalisation + Dropout (30%)
- Layer 3: Dense (64 units, ReLU) + BatchNormalisation + Dropout (30%)
- Output Layer: Dense (6 units, Softmax) for probabilities mapping

6. Numerical Results & Evaluation Report

Upon testing the fully trained MLP on the held-out test split, the classifier achieved world-class mathematical precision. Below are the key overall numerical scores:

Test Set Accuracy

99.8544%

Weighted F1 Score

0.998545

Per-Class Classification Performance

Class	Precision	Recall	F1-Score	Support	OS Gesture Action Map
fist	0.9919	1.0000	0.9959	245	<i>Fist (Play/Pause Media)</i>
l_gesture	1.0000	1.0000	1.0000	159	<i>L Gesture (Open Chrome)</i>
palm	0.9975	0.9975	0.9975	394	<i>Palm (Shift + Tab)</i>
peace	1.0000	0.9982	0.9991	546	<i>Peace (Brightness Up)</i>
three_fingers	1.0000	1.0000	1.0000	640	<i>Three Fingers (Brightness Down)</i>
thumbs_up	1.0000	0.9870	0.9935	77	<i>Thumbs Up (Volume Up)</i>

Confusion Matrix

Actual \ Predicted	fist	l_gesture	palm	peace	three_fingers	thumbs_up
fist	245	0	0	0	0	0
l_gesture	0	159	0	0	0	0
palm	1	0	393	0	0	0
peace	0	0	1	545	0	0
three_fingers	0	0	0	0	640	0
thumbs_up	1	0	0	0	0	76

7. Real-Time Inference & Temporal Smoothing

For seamless interactive control, the system applies two defensive design paradigms to live camera feeds:

1. **Confidence Gating:** Predictions are rejected instantly unless the classification output reaches a minimum threshold of 97% confidence (0.97). This completely stops accidental triggers.
2. **Sliding Window Smoothing:** The pipeline implements a 12-frame history queue. A system action is only dispatched when a single gesture obtains a majority vote of at least 9 frames inside the queue. This dampens flickering transition periods and delivers physical control latency at a stable ~300ms.

8. Exporting for Flutter Mobile Integration

To run native real-time inference on mobile devices, the trained models are compiled into a Float16 TFLite format and exported alongside fully-parameterized coordinate scaling configurations. This guarantees identical dart-side feature scaling, unlocking low-power local predictions on iOS and Android devices.

9. End-to-End Development & Execution Workflow

To reproduce the zero-leakage model training, mobile conversion, and live control deployment, the pipeline executes chronologically across the following seven key stages:

- **Dataset Splits Curation:** Organize raw camera images into isolated split folders (dataset/train/ and dataset/test/) to safeguard physical class separations.
- **Raw Landmark Extraction (process_dataset.py):** Extracts coordinate vectors using MediaPipe, generating independent train_raw.csv and test_raw.csv files with zero initial leakage.
- **Isolated Training Augmentation (augment_data.py):** Performs mirror-flipping, scaling, and noise additions strictly on the training CSV to balance classes, keeping testing landmarks pristine.
- **Leak-Free Normalisation & Splitting (preprocess.py):** Fits standard scaler strictly on the augmented training split, applies geometric transformations, splits validation (15%), and exports preprocessed numpy splits.
- **MLP Model Training & Evaluation (train.py):** Trains the feed-forward MLP with early stopping on validation loss, evaluating the final model on pristine test set arrays.
- **Mobile Edge TFLite Conversion (export_tflite.py):** Quantizes Keras weight matrices to Float16, saving gesture_model.tflite, model_spec.json, and scaler parameters for Dart integration.
- **Live Inference & Client WebSockets (gesture_pipeline.py):** Executes camera frame inference, coordinates feature scaling, filters gestures via majority voting, dispatches system commands, and broadcasts state to Flutter.